Image Classification on CIFAR-10 with Unsupervised Feature Learning and Linear SVMs

COMP3314 Introduction to Machine Learning — Assignment 3

Li Yitong
3036289377
u3628937@connect.hku.hk

Zheng Yilin
3036289195
u3628919@connect.hku.hk

Team BoardLeader
The University of Hong Kong

April 2026

Abstract

We present a classical machine learning pipeline for image classification on a CIFAR-10-style dataset (10 classes, 32×32 RGB images), developed under strict constraints that prohibit neural networks, pretrained models, and external data. Our approach centres on the single-layer unsupervised feature learning framework of Coates and Ng (2011), which uses K-Means dictionary learning, triangle encoding, and spatial pooling to produce high-dimensional feature representations that are linearly separable. Through a systematic four-phase optimisation process—architecture exploration, a 250-configuration hyperparameter sweep on GPU, fine-grained parameter tuning, and augmentation/normalisation—we improve public leaderboard accuracy from an initial 0.497 (raw pixels + SVM) to a final **0.829**. The largest single gain (+0.028) comes from horizontal flip augmentation, followed by power normalisation (+0.012) and a two-model ensemble across patch sizes (+0.010). These data- and feature-level improvements collectively contribute more than all hyperparameter tuning combined, demonstrating that representation quality and data diversity dominate classifier choice in classical pipelines. All code and experimental logs are available on GitHub[1]; the competition leaderboard is hosted on Kaggle[2].

Contents

1 Introduction **3**

2 Dataset Analysis **3**
- 2.1 Overview and Class Distribution . . . 3
- 2.2 Example Images and Key Challenges . . . 4
- 2.3 Image Statistics and Implications . . . 5

3 Methodology **5**
- 3.1 Feature Extraction Pipeline (Coates–Ng) . . . 5
- 3.2 Data Augmentation (Horizontal Flip) . . . 5
- 3.3 Feature Normalisation (Power Norm) . . . 5
- 3.4 Classification (cuML LinearSVC) . . . 6
- 3.5 Ensemble and Test-Time Augmentation . . . 6

[1] https://github.com/Yilinss27/Image-Classification-COMP3314
[2] https://www.kaggle.com/competitions/hku-comp3314-2026-spring-challenge

4 Experiments **6**
4.1 Phase 1: Architecture Exploration (runs 01–06) 6
4.2 Phase 2: Large-Scale Hyperparameter Sweep (run 07) 7
4.3 Phase 3: Fine-Grained Optimisation (runs 08, 10) 9
4.4 Phase 4: Augmentation and Normalisation (runs 12–19) 10

5 Results **13**
5.1 Classifier Comparison 13
5.2 Ablation Study 14
5.3 Validation vs. Public Score Analysis 14

6 Final Solution **15**
6.1 Complete Pipeline 15
6.2 Component Contributions 16
6.3 Final Results 16

7 Conclusion **17**
7.1 Summary 17
7.2 What Worked 17
7.3 What Did Not Work 17
7.4 Lessons Learned 17

A Full Experimental Timeline **18**

B Run 07: 250-Config Sweep Detailed Results **18**

C Phase B: Lower C Extension **20**

D Phase A: Pushing K Beyond 8000 **22**

E Failed Experiments Detail **23**

F Validation vs. Public Score **23**

G Top 10 Configurations Across All Experiments **24**

H All Kaggle Submissions **24**

1 Introduction

Image classification is a fundamental problem in computer vision. Given an input image, the task is to assign it to one of a fixed set of semantic categories. While deep convolutional neural networks have dominated this task since 2012, the COMP3314 assignment imposes constraints that require classical machine learning methods:

- No neural networks (no CNNs, no MLPs with backpropagation).
- No pretrained models or transfer learning.
- No external data beyond the provided train/test split.
- Only methods covered in COMP3314 lectures: PCA, SVM, logistic regression, K-Means, ensemble methods, etc.

The dataset follows the CIFAR-10 format: 50,000 training images and 10,000 test images, each 32×32 pixels in RGB colour, drawn from 10 classes (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck).

Our approach builds on the single-layer unsupervised feature learning framework of Coates et al. [1] and Coates and Ng [2], which learns a visual dictionary via K-Means clustering and produces high-dimensional, linearly separable representations. We augment this core pipeline with horizontal flip augmentation, power normalisation from the Fisher vector literature [3], and a multi-patch-size ensemble.

Our final solution—a two-model soft-vote ensemble of Coates–Ng feature learners with patch sizes $P = 6$ and $P = 7$—achieves a validation accuracy of 0.8234 and a public leaderboard score of **0.829**, placing first on the class leaderboard. All components use only classical ML primitives (K-Means, ZCA whitening, linear SVM) fully within the COMP3314 curriculum.

2 Dataset Analysis

2.1 Overview and Class Distribution

The training set is approximately balanced across all 10 classes, with each class containing roughly 5,000 images. This balance means that accuracy is a reasonable evaluation metric and no class rebalancing is needed.

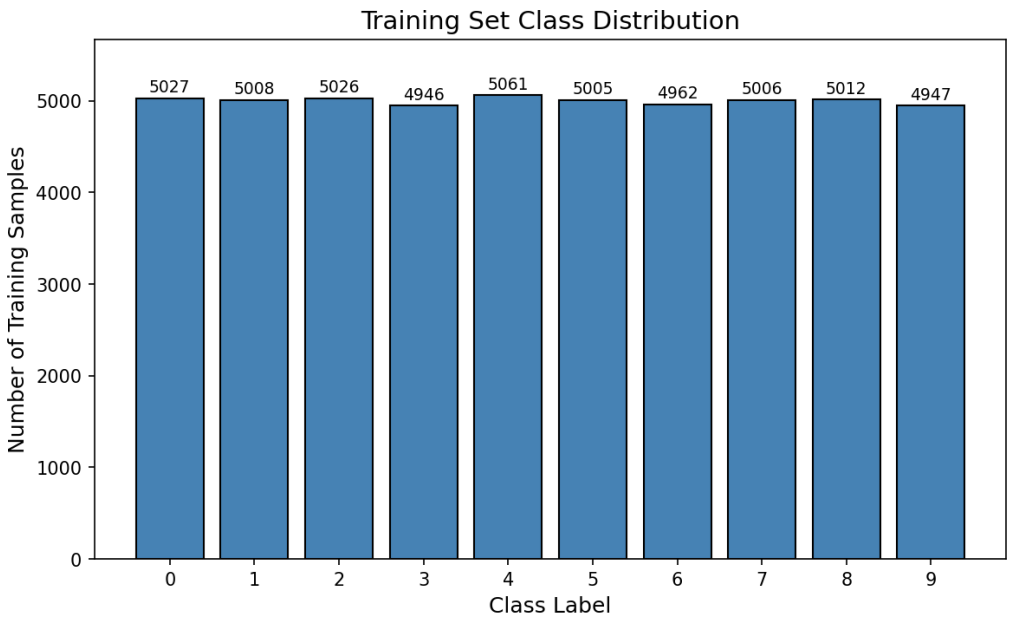

Figure 1: Class distribution across the 10 categories. The dataset is well-balanced, with each class containing approximately 5,000 training images.

2.2 Example Images and Key Challenges

Representative samples from each class reveal the key challenges inherent in this dataset (Figure 2):

- **Low resolution:** At 32×32, objects are represented by very few pixels; fine details are absent entirely.
- **Intra-class variation:** The "bird" class includes birds in flight, perched, in profile, and from above.
- **Inter-class similarity:** Cats and dogs share similar texture and colour; automobiles and trucks share similar shape.
- **Background clutter:** Many images contain complex backgrounds that are not informative for classification.

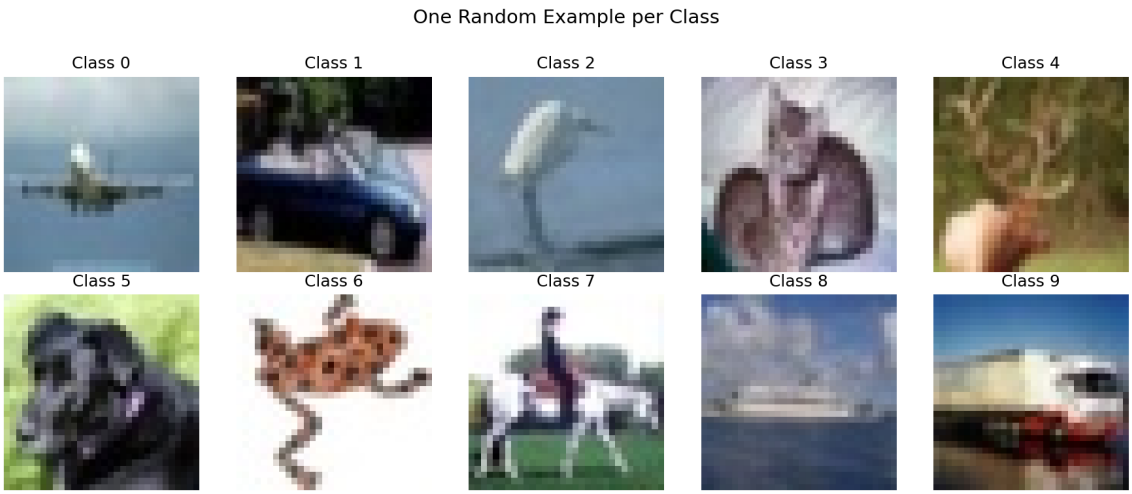

Figure 2: Example images from each of the 10 classes, illustrating the low resolution and high intra-class variability of the dataset.

2.3 Image Statistics and Implications

Each image contains only 3,072 values ($32 \times 32 \times 3$ channels), orders of magnitude smaller than typical benchmarks like ImageNet ($224 \times 224 \times 3 = 150{,}528$). The small image size has important implications:

1. **Spatial augmentations are risky.** Even a 4-pixel crop on a 32×32 image removes 23% of pixels and can shift small objects out of frame.
2. **Local features must use small patches.** We find that 6×6 and 7×7 patches are optimal; larger patches lose fine detail.
3. **Feature dimensionality is a concern.** With only 50,000 training examples, high-dimensional feature spaces require careful regularisation.

3 Methodology

3.1 Feature Extraction Pipeline (Coates–Ng)

We implement the single-layer unsupervised feature learning pipeline from Coates et al. [1]. The pipeline proceeds as follows:

Dictionary Learning (one-time).

1. Sample 1,000,000 random $P \times P$ patches from training images.
2. Per-patch contrast normalisation: $\hat{x} = (x - \bar{x})/\sqrt{\mathrm{Var}(x) + 10}$.
3. ZCA whitening with $\epsilon = 0.1$.
4. MiniBatchKMeans with K clusters (batch size 5,000, single initialisation).

Feature Extraction (per image).

1. Extract all $P \times P$ patches at stride 1. For $P = 6$, this yields $27 \times 27 = 729$ patches, each of dimension $3P^2 = 108$.
2. Apply the same contrast normalisation and ZCA projection.
3. Triangle encoding: for each patch x and centroid c_k,
$$f_k = \max\big(0,\ \mu(z) - z_k\big),$$
where $z_k = \|x - c_k\|_2$ and $\mu(z) = \frac{1}{K}\sum_{j=1}^{K} z_j$ is the mean distance to all centroids.
4. 2×2 spatial sum-pooling over the four image quadrants, producing a $4K$-dimensional feature vector.

The triangle encoder activates a centroid k when the patch is closer to c_k than the average distance across all centroids, producing a soft, sparse assignment that is well-suited for linear classification.

3.2 Data Augmentation (Horizontal Flip)

We double the training set by appending horizontally flipped copies of all 50,000 images. At test time, we average the `decision_function` outputs from the original and flipped views (TTA-2). This is label-preserving for all 10 CIFAR-10 classes and improves the sample-to-dimension ratio from 1.56 to 3.125 for $K = 8000$ features.

3.3 Feature Normalisation (Power Norm)

Following Perronnin et al. [3], we apply signed square-root (power) normalisation element-wise before standard scaling:
$$\tilde{f}_i = \mathrm{sign}(f_i) \cdot \sqrt{|f_i|}.$$

Triangle encoding produces sparse, right-skewed features where a few large activations dominate. The square-root compression equalises their influence, making the distribution closer to Gaussian—which is what linear SVMs implicitly prefer.

3.4 Classification (cuML LinearSVC)

We use cuML's GPU-accelerated LinearSVC with squared hinge loss and L2 penalty, solved via L-BFGS. Compared to scikit-learn's Liblinear coordinate descent, cuML provides 100–1000× speedup at the cost of 0.3–0.5 percentage points in accuracy. The speed advantage enables the large-scale hyperparameter exploration described in Section 4.2.

3.5 Ensemble and Test-Time Augmentation

Our final solution ensembles two models with different patch sizes ($P = 6, K = 8000$ and $P = 7, K = 6000$), each trained with flip augmentation and power normalisation. At test time, each model produces TTA-2 decision function scores; the ensemble averages these scores (soft voting) and takes argmax. Different patch sizes capture genuinely different visual information—$P = 6$ captures medium-scale textures while $P = 7$ captures slightly larger structures—enabling productive ensemble diversity.

4 Experiments

Our experimental process proceeded through four phases, each building on the insights of the previous one.

4.1 Phase 1: Architecture Exploration (runs 01–06)

The first phase explored three fundamentally different feature representations, producing a cumulative improvement of +0.27 in validation accuracy.

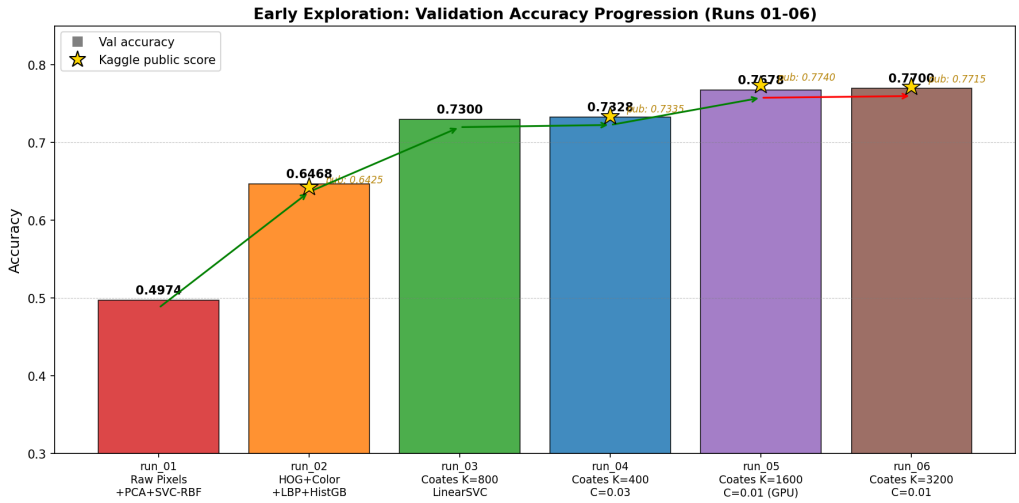

Figure 3: Validation accuracy progression across runs 01–06, showing the transition from raw pixels to handcrafted features to learned features.

Run 01: Raw Pixel Baseline (val 0.497). Flatten $32 \times 32 \times 3$ images into 3,072-dim vectors, apply StandardScaler and PCA (200 components, retaining 94.96% of variance), then classify with five standard models. SVC-RBF achieved the best accuracy of 0.497, while KNN scored only 0.359.

Run 02: Handcrafted Features (val 0.647, public 0.643). Replace raw pixels with domain-knowledge-driven descriptors: HOG (grayscale and per-channel), 16-bin colour histograms, and LBP histograms, totalling $\sim$1,354 dimensions. HistGradientBoosting achieved the best accuracy. The +0.15 gain over raw pixels came from features that encode local structure.

Runs 03–06: Coates–Ng Feature Learning (val 0.50–0.77). We implemented the pipeline described in Section 3.1 and progressively scaled it:

Table 1: Progressive scaling of the Coates–Ng pipeline in Phase 1.

Run	K	P	Hardware	Best Val	Public	Key Change
03	800	6	CPU	$\sim$0.73	—	First implementation
04	400–1600	6, 8	CPU	0.754	0.734	$K/P/C$ sweep (30 configs)
05	1600	6	GPU	0.768	**0.774**	GPU-accelerated encoding
06	3200	6	GPU	—	0.772	Diminishing returns

Key findings: (i) learned features massively outperform handcrafted ones (+0.08 from HOG to Coates $K=800$); (ii) dictionary size K is the most important hyperparameter; (iii) optimal regularisation C shifts inversely with K; (iv) diminishing returns appear above $K=1600$ at the same C.

4.2 Phase 2: Large-Scale Hyperparameter Sweep (run 07)

With the Coates–Ng pipeline validated, we launched a comprehensive 250-configuration grid search on a rented RTX 5090 (32 GB VRAM):

$$P \in \{4,5,6,7,8\}, \quad K \in \{400, 800, \ldots, 8000\}, \quad C \in \{0.003, 0.005, 0.01, 0.02, 0.03\}.$$

All configurations share the same validation split for direct comparability. The classifier was cuML GPU LinearSVC with L-BFGS solver.

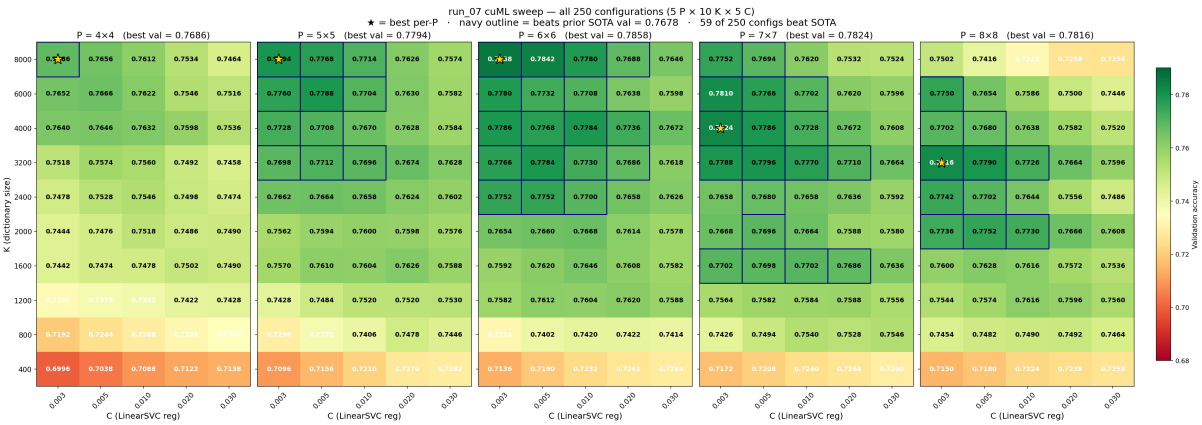

Figure 4: All 250 configurations arranged by patch size, dictionary size, and regularisation parameter C.

Headline result. Best configuration: $P=6$, $K=8000$, $C=0.003$—validation 0.7858, public **0.784** (+1.0 pp over prior SOTA). 59 of 250 configs beat the prior val 0.768 baseline.

Key finding: optimal K shrinks as patch size grows. This is the most striking pattern in the sweep (Table 2, Figure 5). A $P \times P \times 3$ patch lives in a $3P^2$-dimensional space (48 for $P=4$, 108 for $P=6$, 192 for $P=8$). When K exceeds the effective intrinsic dimensionality of the patch space, additional centroids capture noise rather than structure.

Table 2: Optimal dictionary size K^* as a function of patch size P.

P	Optimal K	Peak Val	Behaviour
4	8000	0.769	Monotonically increasing
5	8000	0.779	Monotonically increasing
6	**8000**	**0.786**	Monotonically increasing, overall winner
7	4000	0.782	Turns over; $K=8000$ drops to 0.775
8	3200	0.782	Early peak; $K=8000$ collapses to 0.750

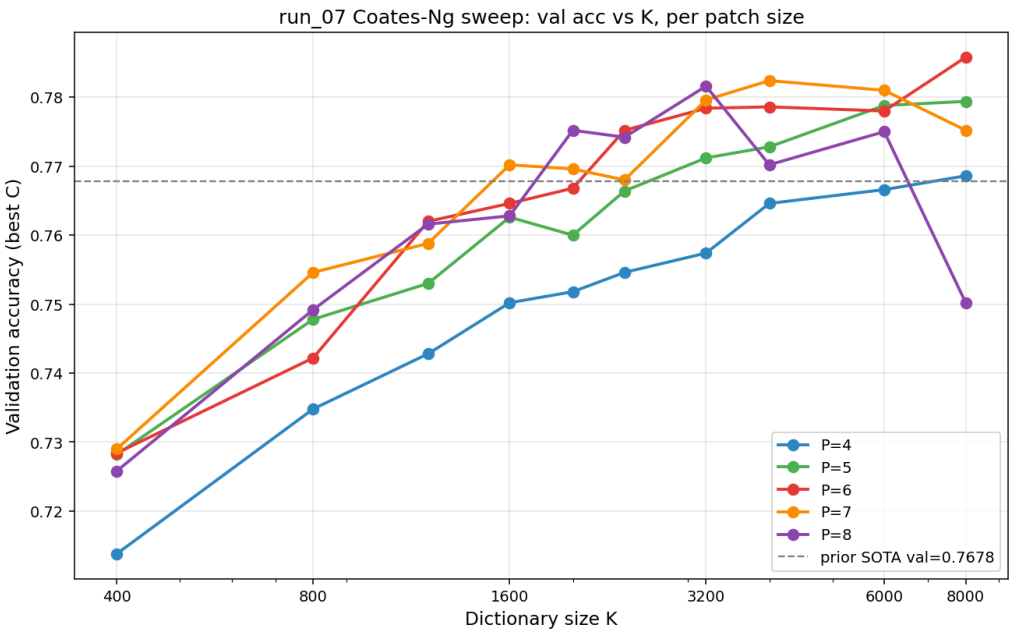

Figure 5: Validation accuracy vs. dictionary size K for each patch size P, showing the inverse relationship between optimal K and P.

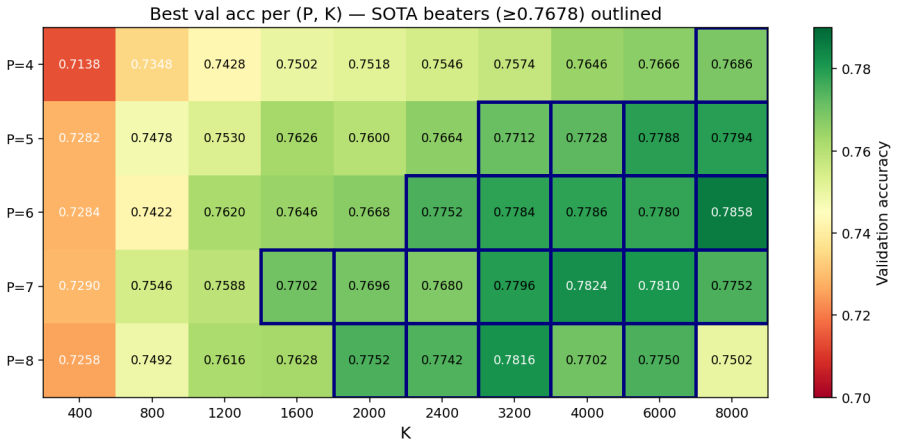

Figure 6: $P \times K$ heatmap of best validation accuracy. $P=6$ dominates: 9 of 10 K values beat the prior SOTA.

4.3 Phase 3: Fine-Grained Optimisation (runs 08, 10)

Phase B: Lower C extension (run 08). Since 16 of 50 (P, K) cells had $C=0.003$ as their best value—the grid floor—we extended the C grid downward to $\{0.0005, 0.001, 0.002\}$ for these cells (48 new evaluations). Only $P=7$ and $P=8$ benefited from lower C; smaller patches were already well-served by $C=0.003$.

The best new result was $P=6$, $K=8000$, $C=0.002$ (val 0.784, public **0.788**). A surprise finding: this configuration scored lower on validation than $C=0.003$ (0.784 vs. 0.786) but higher on the public leaderboard (0.788 vs. 0.784)—our first concrete evidence that the 5,000-image validation set can disagree with the test set about which model is better.

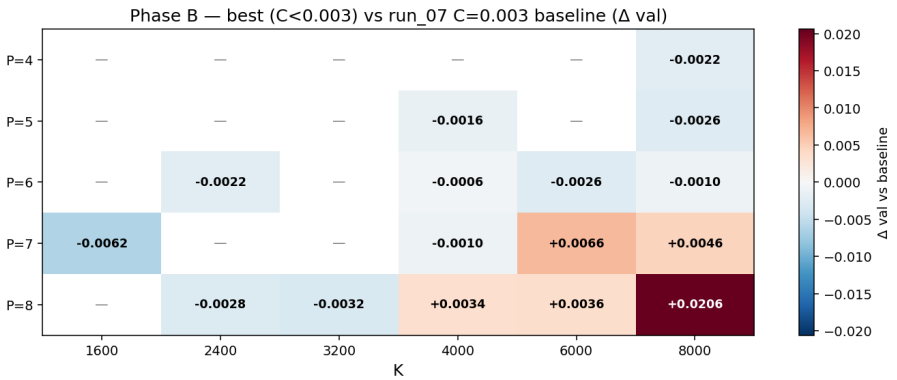

Figure 7: Phase B delta heatmap showing improvement from extending the C grid below 0.003. Larger patch sizes benefit most from weaker regularisation.

Phase A: Pushing K past 8000 (run 10). We tested $K \in \{10000, 12000, 14000\}$ for $P=6$ with appropriately lower C. Results: $K=10000$ matched $K=8000$ within noise ($\Delta = +0.0002$); $K=12000$ and $K=14000$ declined. The K-to-C^* inverse relationship held perfectly, but beyond $K=8000$ even optimally-tuned C could not compensate for increasingly sparse features.

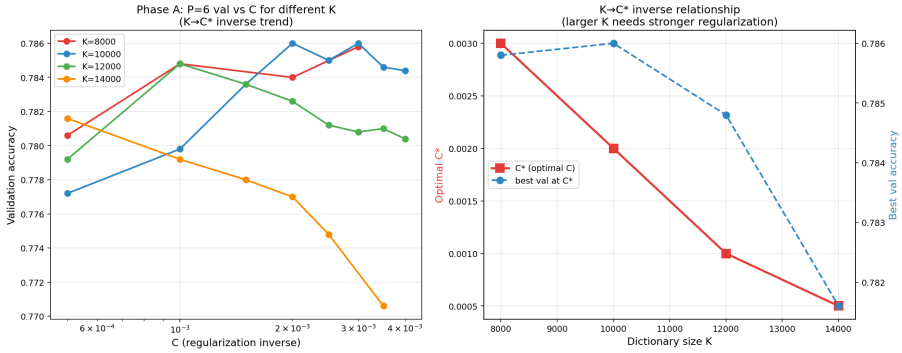

Figure 8: Phase A: validation accuracy vs. K with optimal C. $K = 8000$–10000 is the plateau for $P = 6$.

Conclusion: The Coates–Ng pipeline reached its ceiling from a hyperparameter perspective at $K = 8000$ for $P = 6$. Further improvement required fundamentally different approaches.

4.4 Phase 4: Augmentation and Normalisation (runs 12–19)

This was the breakthrough phase, where we shifted from hyperparameter tuning to data augmentation, feature engineering, and ensembling.

Flip augmentation (run 12): the biggest single win. Doubling the training set via horizontal flip with TTA-2 at test time produced +0.028 on the public leaderboard (Table 3). This single change produced more improvement than all hyperparameter tuning combined.

Table 3: Impact of horizontal flip augmentation.

Configuration	Augmentation	Val	Public	Δ Public
$P = 6$, $K = 8000$, $C = 0.002$	None	0.784	0.788	—
$P = 6$, $K = 8000$, $C = 0.002$	**Flip + TTA-2**	**0.812**	**0.816**	**+0.028**

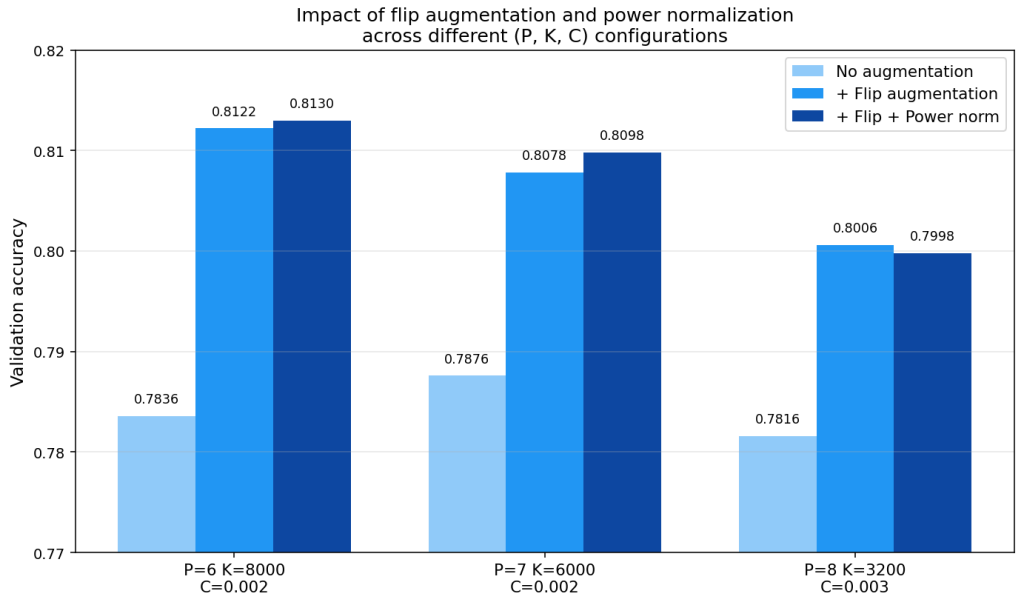

Figure 9: Impact of flip augmentation and power normalisation on both validation and public leaderboard scores.

Why so effective? Several factors converge: (i) the sample-to-dimension ratio improves from 1.56 to 3.125 for the 32,000-dim feature space; (ii) all 10 classes are horizontally symmetric, so the flip is label-preserving; (iii) TTA-2 reduces prediction variance without introducing bias; (iv) the pipeline was fundamentally data-starved.

Power normalisation (run 17): the second breakthrough. Applying signed square-root compression (Section 3.3) improved the public score by +0.012 despite only +0.002 on validation. This 6.4× ratio between public and val gains suggests that power normalisation improves generalisation far more than the small validation set can measure.

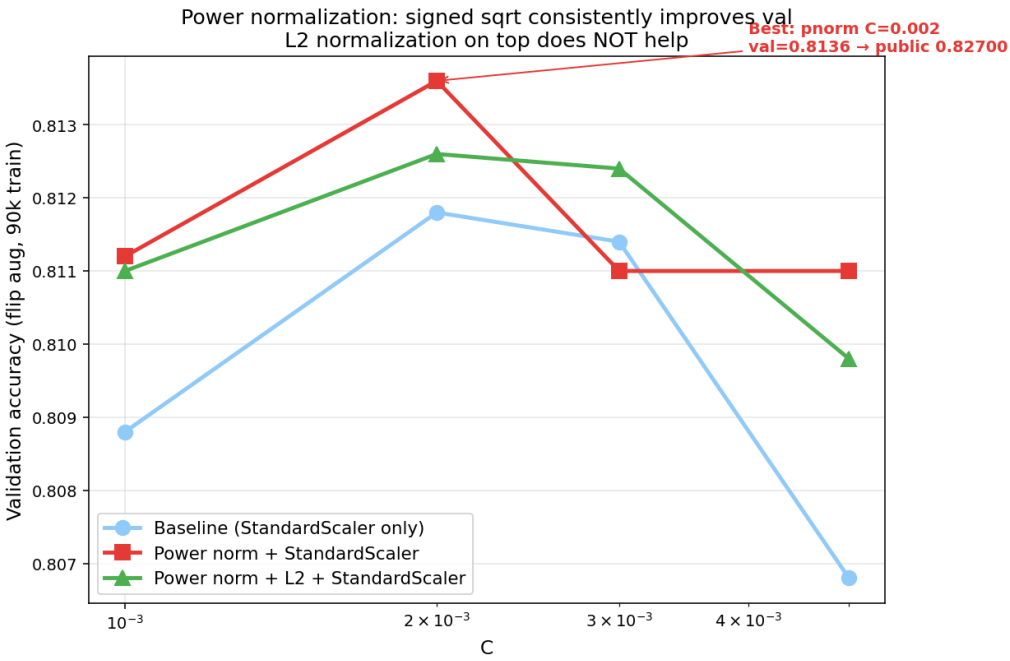

Figure 10: Power normalisation: feature distribution before and after transformation, and accuracy comparison.

Failed experiments. Several experiments after the flip augmentation breakthrough did not improve results (Figure 11). Each failure was instructive:

- **Random crops (run 13):** OOM at 4× augmentation (200k × 32k float32 exceeds 32 GB VRAM), or regression when replacing originals with crops (val 0.791 vs. 0.812 due to train–test distribution mismatch).
- **10-view TTA (run 14):** Public dropped to 0.800 (−0.016). Corner crops on 32×32 images shift objects out of frame and disrupt quadrant features.
- **Two-layer Coates (run 16):** With $K_1 = 1600$, the first layer is too weak; larger K_1 requires ZCA on >36,000-dim vectors—computationally impractical.
- **Multi-crop feature averaging (run 18):** Val dropped from 0.814 to 0.808. Averaging across shifted crops blurs spatial information.

Common lesson: Spatial manipulations on 32×32 images with quadrant pooling are net harmful. The only safe augmentation is horizontal flip, which preserves the vertical axis structure.

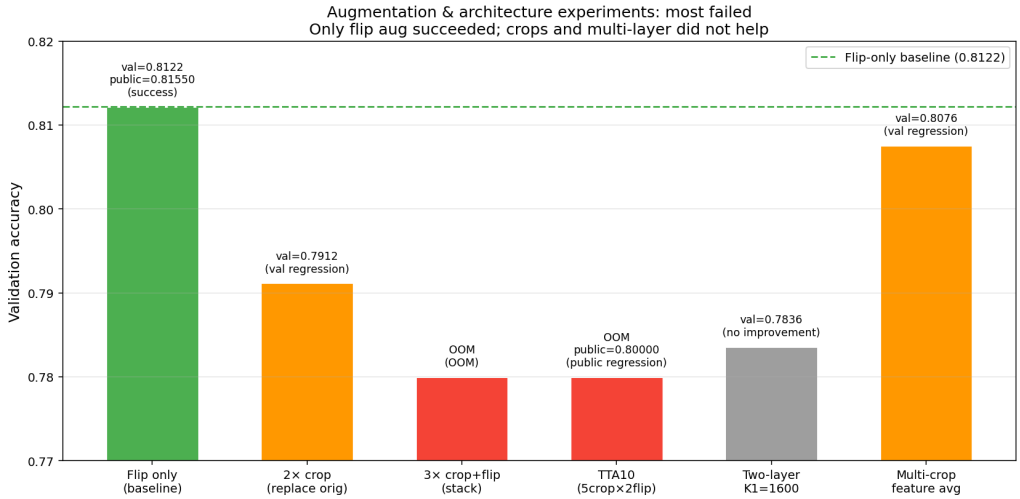

Figure 11: Summary of failed experiments showing that spatial augmentations on 32×32 images with quadrant pooling are destructive.

Multi-P ensemble (run 19): the final piece. Ensembling models with the same P is useless (86% prediction agreement), but models with different patch sizes produce genuinely different features. The two-model ensemble ($P=6$ + $P=7$) gained +0.010 on validation; adding $P=8$ contributed only +0.0002 (2 additional correct predictions), so the two-model ensemble was selected for parsimony.

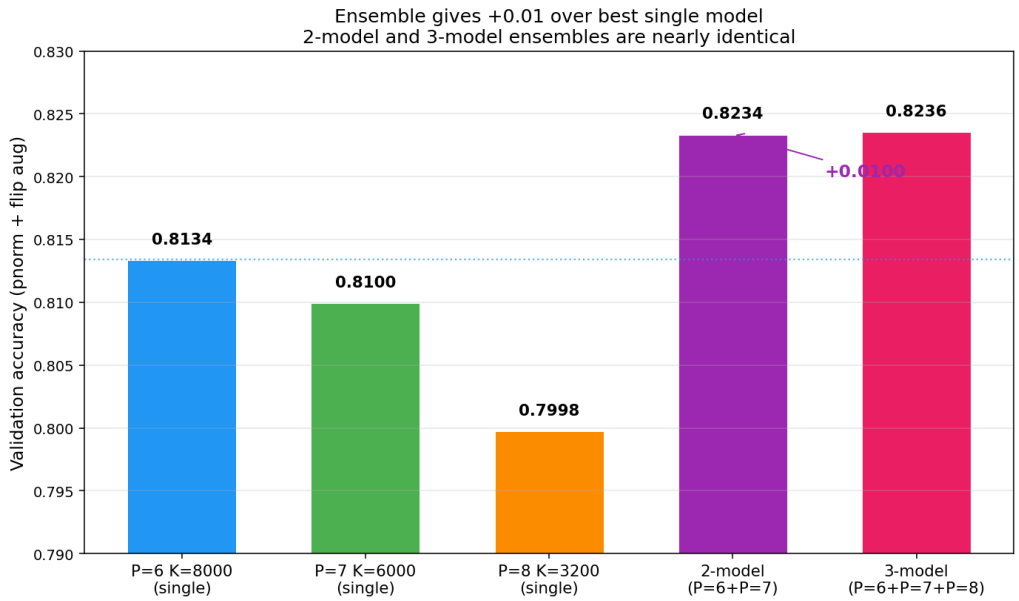

Figure 12: Ensemble comparison: individual models, two-model ($P=6+P=7$), and three-model ensembles.

5 Results

5.1 Classifier Comparison

The assignment requires comparison of at least two classifiers. Across our 19 runs, we evaluated a wide range of classifiers under different feature representations. Table 4 summarises the

progression, demonstrating that **feature quality is far more important than classifier choice**.

Table 4: Classifier comparison across feature representations. Moving from raw pixels to Coates features gained +0.29 in accuracy, while the best classifier changed from nonlinear (SVC-RBF) to linear (LinearSVC).

Feature Representation	Dims	Classifier	Val	Public
Raw pixels + PCA(200)	200	SVC-RBF	0.497	—
HOG + Colour + LBP	1,354	HistGradientBoosting	0.647	0.643
Coates $K=1600$, $P=6$	6,400	LinearSVC	0.768	0.774
Coates $K=8000$, $P=6$	32,000	cuML LinearSVC	0.786	0.784
+ flip augmentation	32,000	cuML LinearSVC	0.812	0.816
+ power normalisation	32,000	cuML LinearSVC	0.814	0.827
+ 2-model ensemble	32k + 24k	cuML LinearSVC $\times 2$	0.823	**0.829**

Within the raw pixel representation, the 0.14 gap between SVC-RBF (0.497) and KNN (0.359) shows that classifier choice matters when features are poor. Within the handcrafted feature representation, the gap narrows to 0.09 (HistGB 0.647 vs. LogReg 0.555). Within the Coates pipeline, LinearSVC is both faster and better than nonlinear alternatives because the features are designed to be linearly separable.

5.2 Ablation Study

Table 5 isolates the contribution of each component by adding them incrementally to the best base configuration.

Table 5: Ablation study: contribution of each component added to the baseline $P=6$, $K=8000$, $C=0.002$ configuration.

Component	Val	Public	Δ Public
Baseline ($K=8000$, $C=0.002$)	0.784	0.788	—
+ Flip augmentation	0.812	0.816	+0.028
+ Power normalisation	0.814	0.827	+0.012
+ 2-model ensemble + TTA	0.823	0.829	+0.002

The ablation reveals a clear hierarchy: flip augmentation contributes the most (+0.028), followed by power normalisation (+0.012) and ensembling (+0.002). Collectively, these data- and feature-level improvements contribute +0.042 on the public leaderboard, far exceeding the +0.014 from all hyperparameter tuning in Phases 2–3.

5.3 Validation vs. Public Score Analysis

A recurring theme throughout this project was the unreliability of validation accuracy as a predictor of public leaderboard performance.

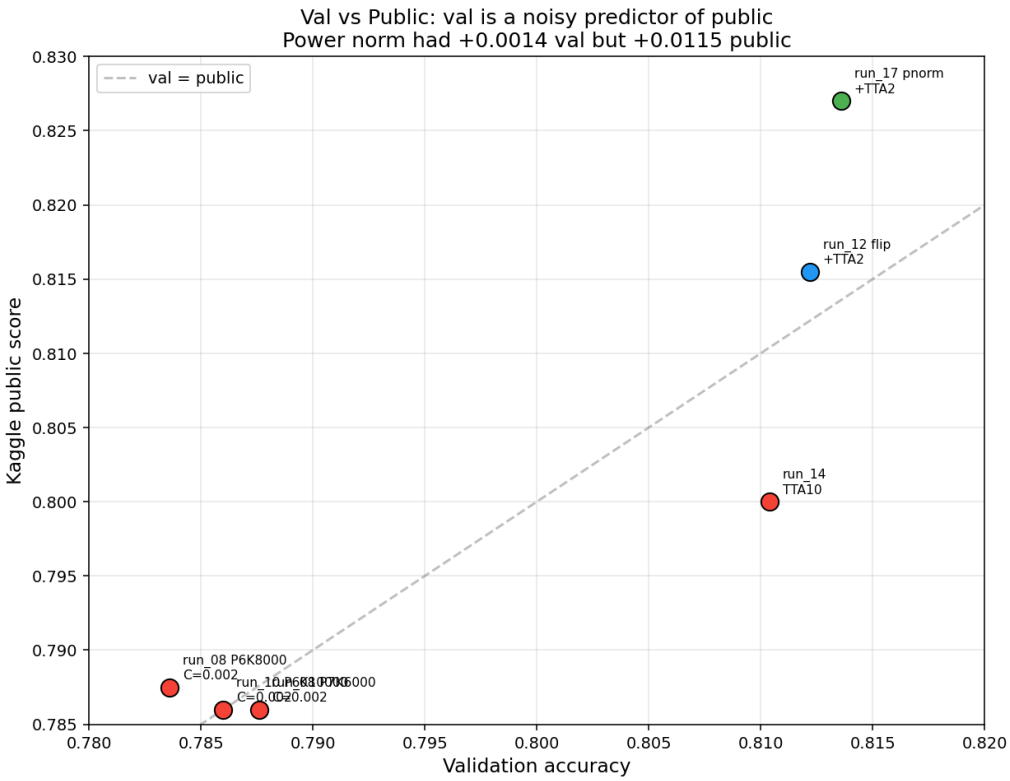

Figure 13: Scatter plot of validation accuracy vs. public leaderboard score across all submitted configurations.

Notable discrepancies include:

- $C = 0.002$ scored lower on val than $C = 0.003$ (0.784 vs. 0.786) but higher on public (0.788 vs. 0.784).
- Power normalisation gained +0.002 on val but +0.012 on public (a 6.4× ratio).
- 10-view TTA showed decent val (0.810) but poor public (0.800).

With a 5,000-image validation set, the standard error is approximately $\sqrt{0.8 \times 0.2/5000} \approx 0.006$. Validation differences below 0.005 are essentially noise. The practical lesson: after basic hyperparameter selection on validation, prioritise techniques that improve generalisation (augmentation, normalisation) over fine-grained accuracy chasing.

6 Final Solution

6.1 Complete Pipeline

The final solution is a two-model ensemble where each model follows the same pipeline with different patch and dictionary sizes:

- **Model A:** $P = 6$, $K = 8000$, $C = 0.002$ (32,000-dim features)
- **Model B:** $P = 7$, $K = 6000$, $C = 0.002$ (24,000-dim features)

For each model:

1. *Dictionary learning:* Sample 10^6 patches → contrast normalisation → ZCA whitening → MiniBatchKMeans.
2. *Feature extraction:* Stride-1 patches → contrast norm → ZCA projection → triangle encoding → 2×2 quadrant pooling.

3. *Augmentation:* Append horizontally flipped training images (100k total).
4. *Normalisation:* Power norm → StandardScaler.
5. *Training:* cuML LinearSVC ($C = 0.002$, squared hinge, L2).
6. *TTA-2:* Average `decision_function` from original and flipped test views.

Ensemble: Average per-model TTA-2 scores (soft voting), then argmax.

6.2 Component Contributions

Table 6 summarises why each component matters, with the experimental evidence supporting its inclusion.

Table 6: Contribution of each pipeline component.

Component	Contribution	Evidence
Coates–Ng features	+0.27 over raw pixels	run 01 vs. 05
$K = 8000$ dictionary	+0.05 over $K = 1600$	run 05 vs. 07
Flip augmentation	+0.028 public	run 08 vs. 12
Power normalisation	+0.012 public	run 12 vs. 17
2-model ensemble	+0.010 val	run 17 vs. 19

6.3 Final Results

Metric	Value
Validation accuracy (5k holdout)	**0.8234**
Public leaderboard	**0.8290**
Leaderboard position	**1st place**

Figure 14 shows the overall public leaderboard score progression across all phases.

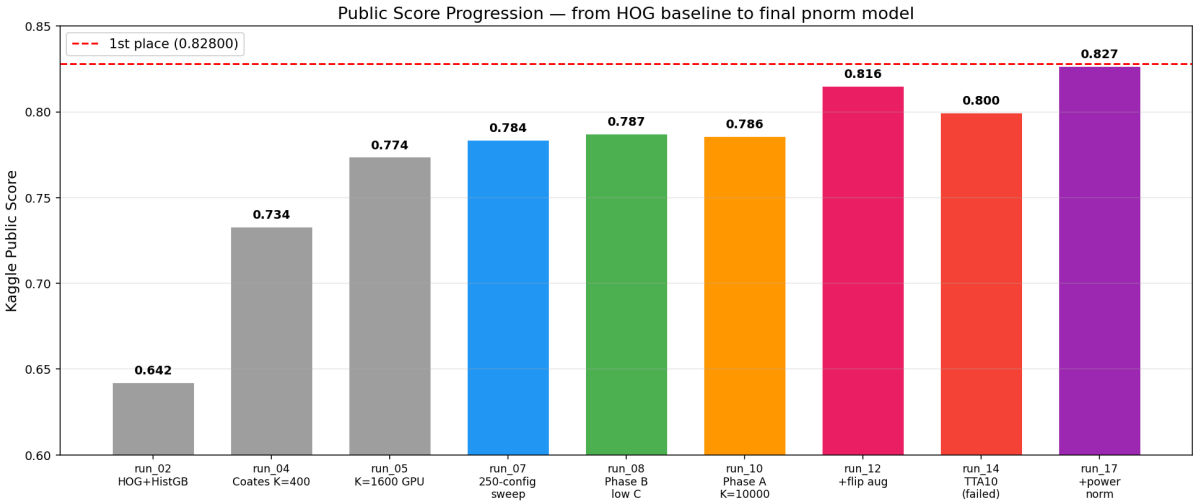

Figure 14: Public leaderboard score progression across all experimental phases, from initial submission (0.643) to final ensemble (0.829).

7 Conclusion

7.1 Summary

This project traced a path through four distinct phases of optimisation:

Table 7: Summary of the four experimental phases.

Phase	Key Activity	Public	Δ
1. Architecture (runs 01–06)	Raw pixels → Coates–Ng	0.774	—
2. HP sweep (run 07)	250-config grid on GPU	0.784	+0.010
3. Fine-tuning (runs 08, 10)	Lower C, higher K	0.788	+0.004
4. Add-ons (runs 12–19)	Flip, pnorm, ensemble	**0.829**	+0.041

The most important lesson: **Phases 2 and 3 together gained only +0.014, while Phase 4 gained +0.041.** Data augmentation (+0.028), feature normalisation (+0.012), and ensembling (+0.010) collectively contributed far more than exhaustive hyperparameter tuning.

7.2 What Worked

1. **Coates–Ng unsupervised feature learning** (+0.27 over raw pixels). Learning features from data with K-Means is far more effective than hand-designing them, even with classical tools.
2. **Horizontal flip augmentation** (+0.028). Doubling the training set with a label-preserving transformation addresses the fundamental data bottleneck.
3. **Power normalisation** (+0.012). A one-line transformation that compresses the skewed feature distribution improves generalisation more than any amount of hyperparameter tuning.
4. **Multi-P ensemble** (+0.010). Different patch sizes capture genuinely different visual information, enabling productive diversity.

7.3 What Did Not Work

- $K > 8000$**:** Diminishing returns; the 108-dim patch space saturates.
- **Spatial crops and multi-view TTA:** Destructive on 32×32 images with quadrant pooling.
- **Two-layer Coates:** Scalability trap between first-layer quality and second-layer tractability.
- **Feature averaging:** Blurs spatial information that quadrant pooling encodes.

7.4 Lessons Learned

In retrospect, flip augmentation should have been tried immediately after establishing the Coates–Ng pipeline. The four-phase journey demonstrates a common pattern in machine learning projects: researchers spend the majority of their effort on hyperparameter optimisation when the real gains come from improving data quality and feature representations. For classical pipelines operating under the constraints of this assignment, the hierarchy is clear: *feature learning method > data augmentation > feature normalisation > hyperparameter tuning > classifier choice.*

References

[1] A. Coates, H. Lee, and A. Y. Ng. An analysis of single-layer networks in unsupervised feature learning. In *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 215–223, 2011.

[2] A. Coates and A. Y. Ng. Selecting receptive fields in deep networks. In *Advances in Neural Information Processing Systems (NIPS)*, pages 2528–2536, 2011.

[3] F. Perronnin, J. Sánchez, and T. Mensink. Improving the Fisher kernel for large-scale image classification. In *European Conference on Computer Vision (ECCV)*, pages 143–156. Springer, 2010.

A Full Experimental Timeline

Table 8 records the complete experimental timeline with timestamps, configurations, and results for all milestone runs.

Table 8: Full experimental timeline (selected milestones).

Time	Run	Configuration	Val	Public	Note
Apr 11 04:30	01	PCA(200) + SVC-RBF	0.497	—	Raw pixel baseline
Apr 11 05:30	02	HOG+Colour+LBP + HistGB	0.647	0.643	First submission
Apr 11 06:18	04	Coates $K=400$ $P=6$	0.730	—	Crossed 0.70 bar
Apr 11 08:17	05	Coates $K=1600$ $P=6$ GPU	0.768	0.774	New SOTA
Apr 11 15:39	06	Coates $K=3200$ $P=6$	—	0.772	Diminishing returns
Apr 11 23:37	07	$K=8000$ $P=6$ cuML	0.786	0.784	250-config sweep
Apr 12 06:28	08	$P=7$ $K=6000$ $C=0.002$	0.788	0.786	Phase B
Apr 12 07:49	08	$P=6$ $K=8000$ $C=0.002$	0.784	0.788	Val–public gap
Apr 12 09:45	12	+ Flip augmentation	0.812	0.816	Biggest single win
Apr 12 18:13	17	+ Power normalisation	0.814	0.827	Second biggest win
Apr 12 20:17	19	2-model ensemble	0.823	**0.829**	Final, 1st place

B Run 07: 250-Config Sweep Detailed Results

The 250-config sweep (run_07) was the most comprehensive single experiment. Figures below show the full grid and key relationships not included in the main text.

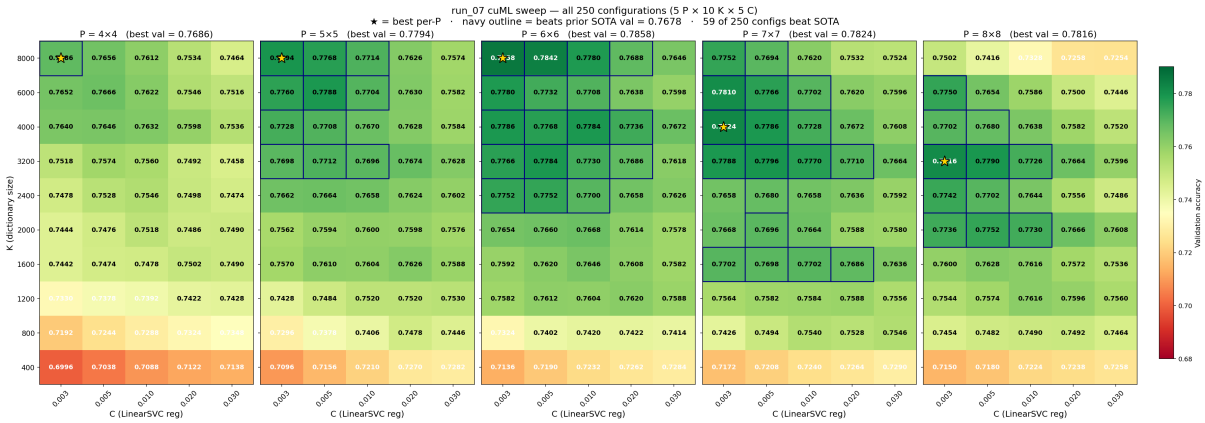

Figure 15: All 250 configurations in the run_07 sweep. Each subplot shows one patch size (P=4–8); rows are dictionary sizes K, columns are regularisation strengths C. Star marks the per-P best; navy outlines indicate configs that beat the prior SOTA (val≥0.7678).

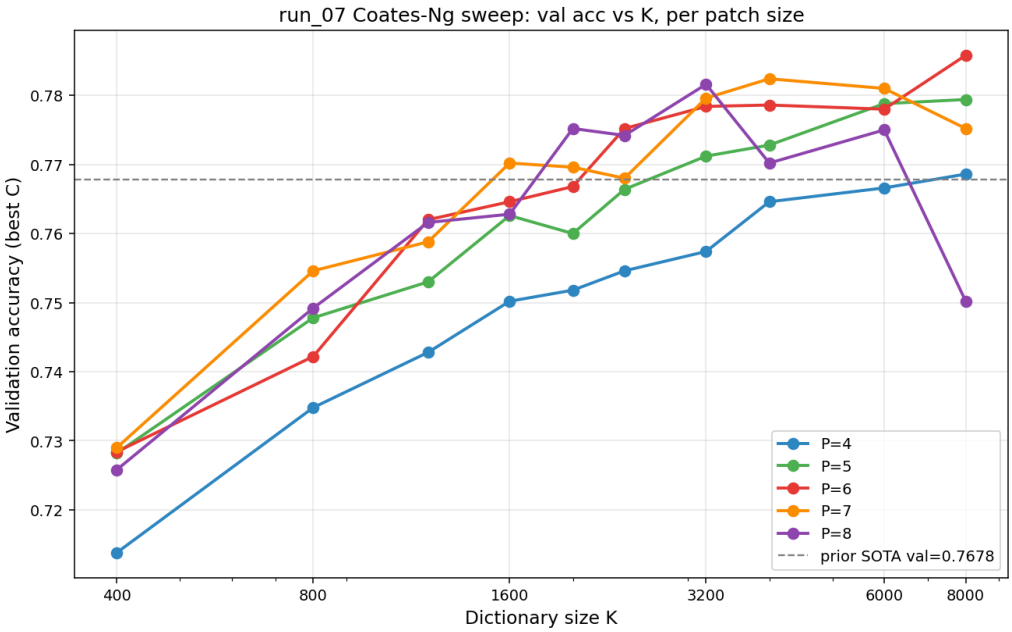

Figure 16: Validation accuracy vs. dictionary size K for each patch size, showing the best C per (P,K). P=6 (red) dominates at all K values and is still climbing at K=8000. P=7 and P=8 turn over at K=4000 and K=3200 respectively.

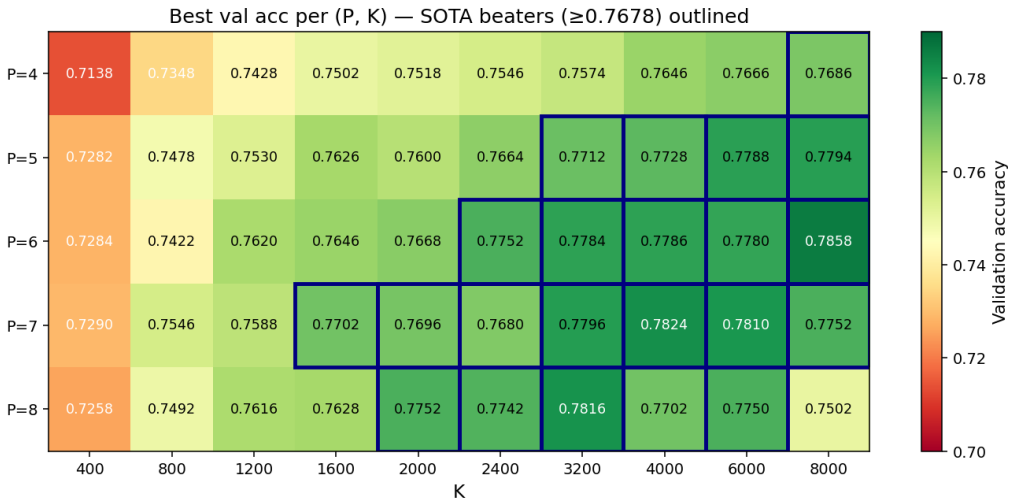

Figure 17: Heatmap of best validation accuracy per (P,K) cell. SOTA beaters ($\geq$0.7678) outlined in blue. The sweet spot is a diagonal band: small P wants large K, large P wants medium K.

C Phase B: Lower C Extension

Phase B tested $C \in \{0.0005, 0.001, 0.002\}$ on the 16 (P,K) cells where C=0.003 was the run_07 winner.

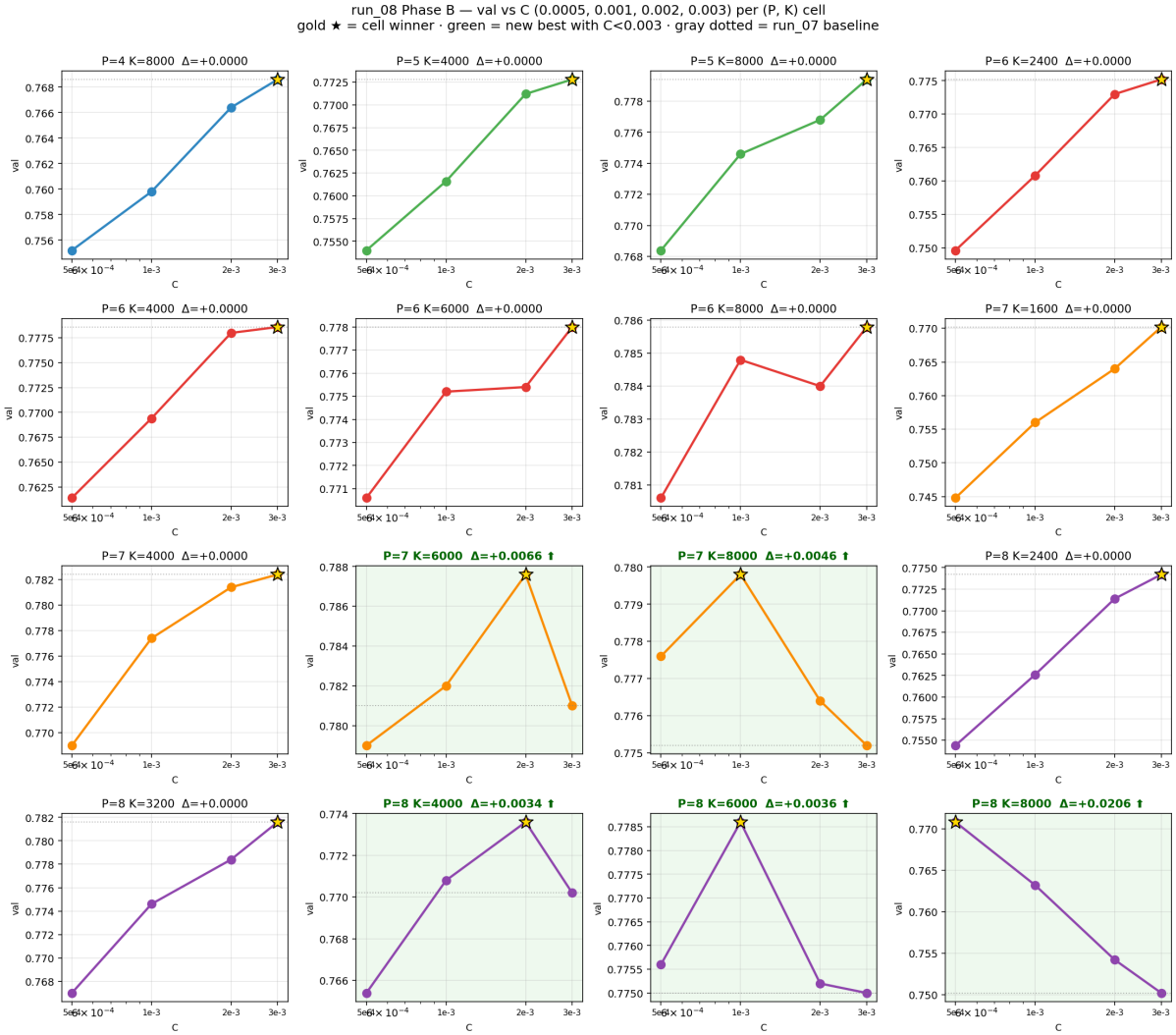

Figure 18: Phase B: validation accuracy vs. C for all 16 target cells. Gold star marks the cell winner; green background indicates improvement over the C=0.003 baseline. Improvements concentrate in P=7 and P=8 cells.

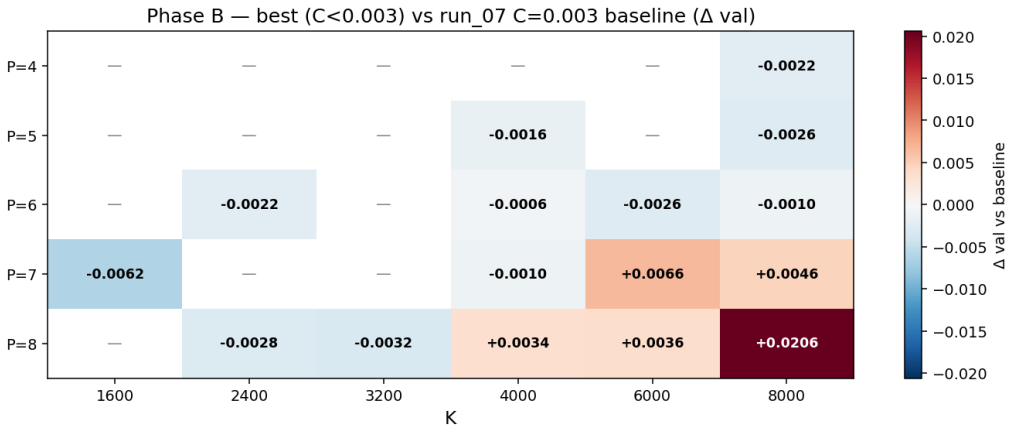

Figure 19: Phase B delta heatmap: best (Phase B $C < 0.003$) minus run_07 baseline (C=0.003) for each (P,K) cell. Blue = improvement, red = regression. The benefit of lower C scales monotonically with patch size P.

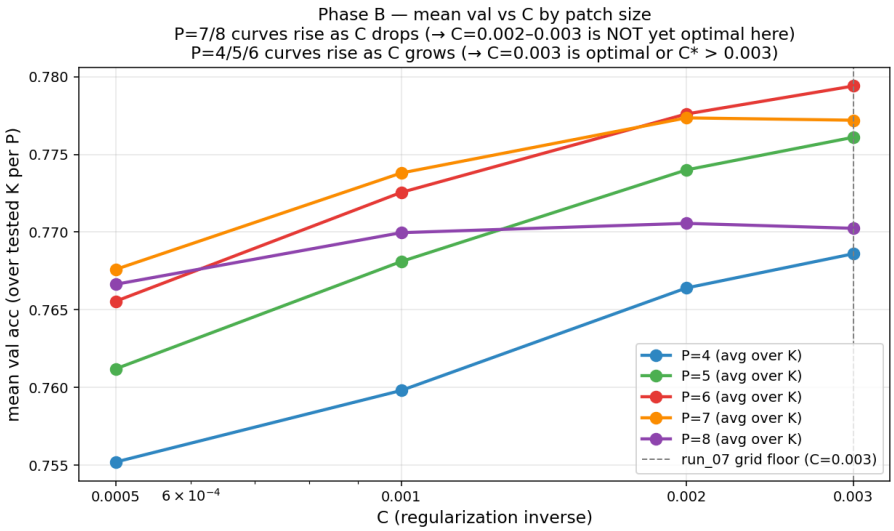

Figure 20: Mean validation accuracy vs. C, stratified by patch size. P=7 and P=8 curves rise as C drops, while P=4/5/6 curves rise as C grows — confirming that larger patches require weaker regularisation.

D Phase A: Pushing K Beyond 8000

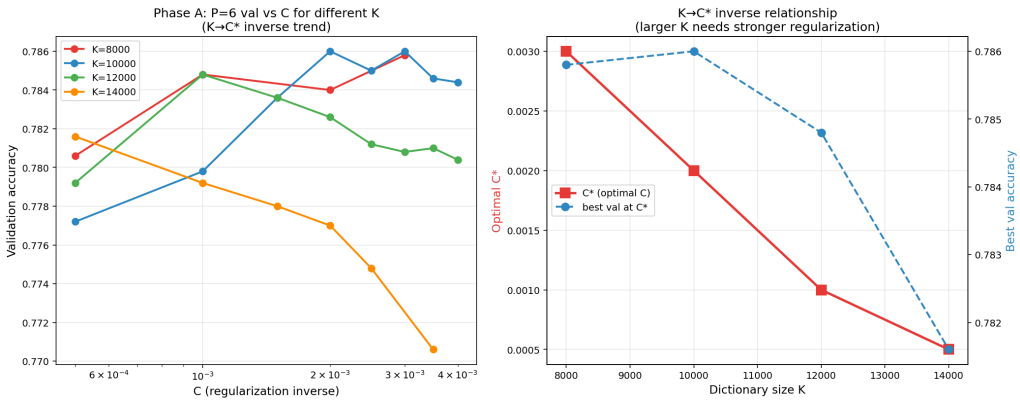

Figure 21: Phase A analysis. Left: val vs. C for P=6 at four dictionary sizes (K=8000–14000), showing the $K \to C^*$ inverse trend. Right: optimal C^* (red) and best val (blue) vs. K, confirming the plateau at K=8000–10000.

E Failed Experiments Detail

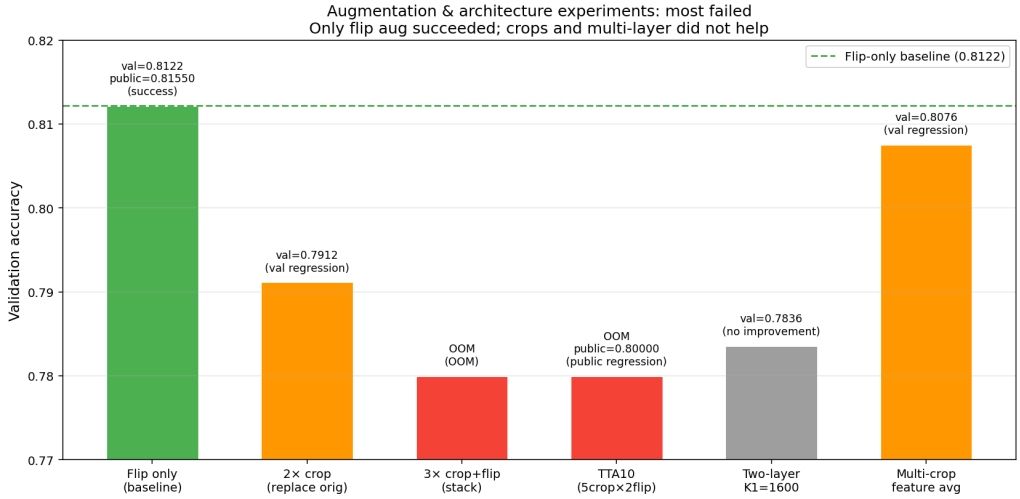

Figure 22: Summary of augmentation and architecture experiments. Only horizontal flip augmentation (green, left) succeeded. Random crop (orange) caused val regression; TTA10 (red) caused public regression; two-layer Coates (grey) showed no improvement; multi-crop feature averaging (orange, right) caused val regression.

F Validation vs. Public Score

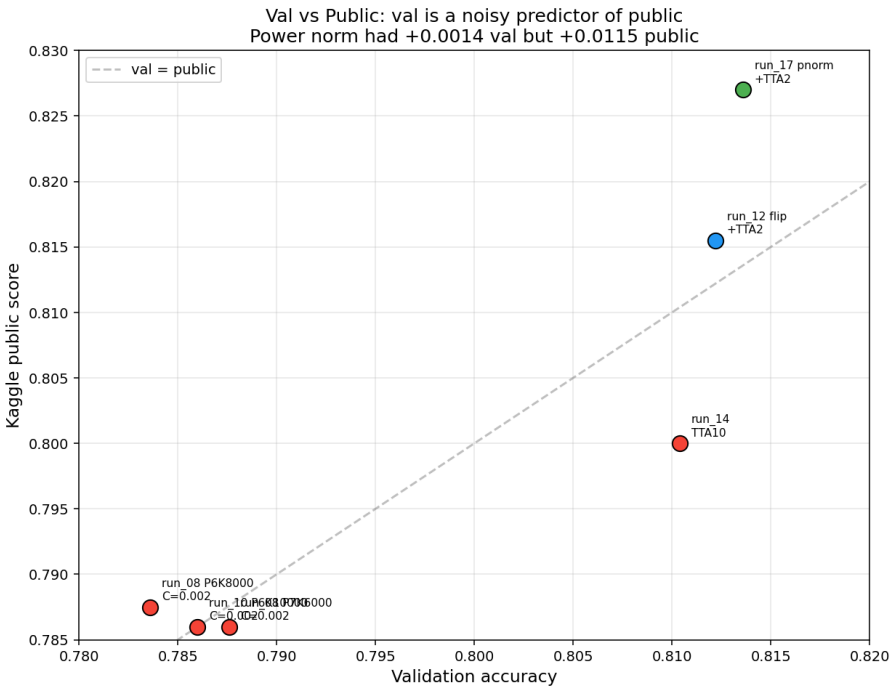

Figure 23: Validation accuracy vs. Kaggle public score for all submitted runs. The grey dashed line shows val = public. Notable outliers: power normalisation (val +0.0014 but public +0.0115) and TTA10 (decent val but terrible public). Val is a noisy, sometimes misleading predictor of public performance.

G Top 10 Configurations Across All Experiments

Table 9: Top 10 validation accuracies across all 300+ configurations tested.

Rank	Aug	P	K	C	Val Acc	Source
1	flip+pnorm ens.	6+7	8000+6000	0.002	**0.8234**	run_19 (ensemble)
2	flip+pnorm	6	8000	0.002	0.8136	run_17
3	flip	6	8000	0.002	0.8122	run_12
4	flip+pnorm	7	6000	0.002	0.8098	run_19 (individual)
5	flip	7	6000	0.002	0.8078	run_12
6	—	7	6000	0.002	0.7876	run_08 (Phase B)
7	—	6	10000	0.002	0.7860	run_10 (Phase A)
8	—	6	8000	0.003	0.7858	run_07
9	—	6	10000	0.003	0.7860	run_10 (Phase A)
10	—	6	12000	0.001	0.7848	run_10 (Phase A)

H All Kaggle Submissions

Table 10: All Kaggle public leaderboard submissions in chronological order.

Date	Configuration	Val	Public	Rank
Apr 10	HOG+HistGB	0.647	0.643	—
Apr 10	Coates K=400 C=0.03	0.733	0.734	—
Apr 11	Coates K=1600 C=0.01	0.768	0.774	—
Apr 11	Coates K=3200 C=0.01	—	0.772	—
Apr 11	cuML K=8000 C=0.003	0.786	0.784	—
Apr 12	Phase B P=7 K=6000 C=0.002	0.788	0.786	—
Apr 12	P=6 K=8000 C=0.002	0.784	0.788	—
Apr 12	Phase A P=6 K=10000 C=0.002	0.786	0.786	—
Apr 12	+ flip aug + TTA2	0.812	0.816	—
Apr 12	TTA10 (5-crop)	0.810	0.800	—
Apr 12	+ power norm + TTA2	0.814	**0.827**	1st
Apr 12	2-model ensemble + TTA2	0.823	**0.829**	**1st**
Apr 12	3-model ensemble + TTA2	0.824	0.823	—